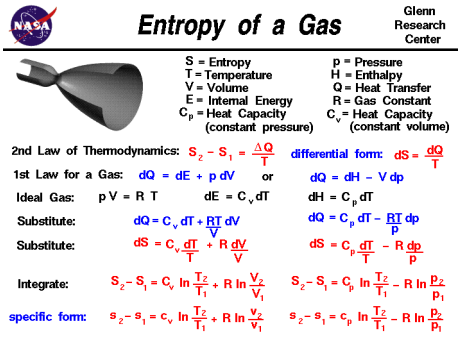
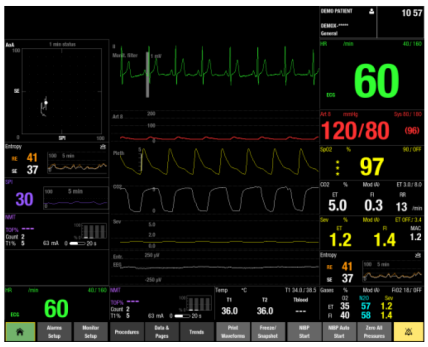
1 Introduction

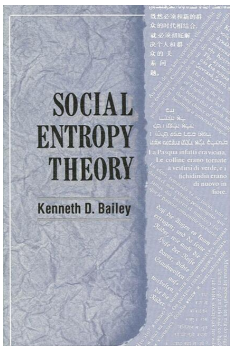
"Entropy" is a very overloaded term with a range of different meanings across and within different domains of math and science.



(a) Thermodynamic entropy of a gas [1]



(b) EEG entropy on a patient monitor [2]



(c) Social entropy [3]

Figure 1: A few of the many things called "entropy".

The link between many of these is more spiritual than anything else: entropies "quantify uncertainty and information". Today, we'll focus on entropy in the context of information theory. That is, how can we mathematically model uncertainty and the transfer of information?

2 Shannon entropy

Let's start with an exercise from [4] that describes pretty well how the mathematical formula for entropy was discovered. Starting with the desired properties, we can derive a derive that entropy should take a given form:

> **Exercise 11.2: (Intuitive justification for the definition of entropy)** Suppose we are trying to quantify how much information is provided by an event E which may occur in a probabilistic experiment. We do this using an 'information function' $I(E)$ whose value is determined by the event E. Suppose we make the following assumptions about this function:
>
> (1) $I(E)$ is a function only of the probability of the event E, so we may write $I = I(p)$, where p is a probability in the range 0 to 1.
>
> (2) I is a smooth function of probability.
>
> (3) $I(pq) = I(p) + I(q)$ when $p, q > 0$. (*Interpretation:* The information gained when two independent events occur with individual probabilities p and q is the sum of the information gained from each event alone.)
>
> Show that $I(p) = k \log p$, for some constant, k. It follows that the average information gain when one of a mutually exclusive set of events with probabilities $p_1, \ldots, p_n$ occurs is $k \sum_i p_i \log p_i$, which is just the Shannon entropy, up to a constant factor.

Figure 2: Exercise 11.2 from [4].

This is similar to how Claude Shannon invented Shannon entropy in his seminal 1948 paper "A Mathematical Theory of Communication" [5]. We define Shannon entropy as follows:

Definition 2.1: Shannon entropy

For a discrete, random variable $X = 1, ..., n$ with probability distribution $p_1, ..., p_n$, the Shannon entropy is defined as:

$$H(X) \equiv H(p_1, ..., p_n) \equiv -\sum_x p_x \log p_x$$

There are two parts of this formulation which are particularly ambigious.

First, we need to make the exception that if $p_x = 0$, then we neglect the term by agreeing $0 \log 0 \equiv 0$. Intuitively, if $p_x = 0$, then X can never take the value of x, so why should this case affect the entropy!

Second, what is the base of the logarithm? We will see later that taking logarithms in base 2 allows for a meaningful connection to the number of bits needed to represent data pertaining to X in binary. This is part of the reason why bits is said to be the unit for $H(X)$. That is, we say X has $H(X)$ *bits of entropy.*

Something else to notice is that the entropy $H(X)$ does not depend on the labels of what X can take, and is only a function of its distribution.

Let's play with this a little bit to get a feel for the formula:

Exercise 2.2: Playing with Shannon entropy

1. What is the entropy associated with the toss a fair coin?
2. What is the entropy associated with the roll of a fair die?
3. What is the entropy associated with the roll of a *Mario die* (with sides 1,3,3,3,5,6)? What about a *Peach die* (0,2,4,4,4,6)?
4. How do the above compare to a fair 4-sided die?

So we understand how to calculate Shannon entropy, but what is the intuition behind it representing a quantity of information? One answer is that it captures *the physical resources needed to communicate information.*

Suppose we have an information source, which is constantly producing i.i.d. discrete random variables $X_1, X_2, ...$ What is the expected amount of minimal physical ressources we need to send, on average, so that this string of random variables can be perfectly reconstructed by the receiver? The answer, amazingly, is $H(X) = H(X_1) = H(X_2)...$ bits!

Let's consider a simple example (from [4]). Consider a source of information that produces a sequence of four signals: $X = A, B, C,$ or D. Now suppose that the sources wanted to send this sequence to some receiver by encoding the symbols into binary. We know that we can build an encoding scheme that uses 2 bits to represent each symbol:

Symbol	A	B	C	D
Encoding	`00`	`01`	`10`	`11`

But now suppose we know that these symbols aren't generated uniformly, and that we know the distribution:

Symbol	A	B	C	D
Probability	1/2	1/4	1/8	1/8

Then we can come up with an encoding scheme that leverages this:

Symbol	A	B	C	D
Probability	1/2	1/4	1/8	1/8
Encoding	`1`	`10`	`110`	`111`

This scheme yeilds an expected number of bits per symbol to be

$$1\left(\frac{1}{2}\right)+2\left(\frac{1}{4}\right)+3\left(\frac{1}{8}\right)+3\left(\frac{1}{8}\right)=\frac{7}{4}.$$

Sure enough, if we calculate the entropy of the distribution of symbols:

$$\begin{aligned}-\frac{1}{2}\log\left(\frac{1}{2}\right)-\frac{1}{4}\log\left(\frac{1}{4}\right)-\frac{1}{8}\log\left(\frac{1}{8}\right)-\frac{1}{8}\log\left(\frac{1}{8}\right)&\\ =\frac{1}{2}\cdot 2+\frac{1}{4}\cdot 2+\frac{1}{8}\cdot 3+\frac{1}{8}\cdot 3&\\ =\frac{7}{4}&\end{aligned}$$

So this isn't just a good encoding scheme, its the best we can possibly do!

Lets give a quick, semi-formal proof of this optimality (following from [6]). Suppose that we want to transmit some large, but finie sequence of N symbols $X=1,2,...,m$ drawn i.i.d. with probabilities $P(X=i)=p(i)$. Because each symbol is drawn i.i.d, the probability of getting any one sequence $\bar{x}=(x_1,...,x_N)$ is

$$p(\bar{x})=p(x_1)p(x_2)\cdots p(x_N)$$

We can collect terms which have the same symbol, where $n(i)$ is the number of symbols in $\bar{x}$ which are i:

$$p(\bar{x})=\prod_{i=1}^{m}p(i)^{n(i)}$$

By the law of large numbers, we know that as $N\to\infty$, we can use the probability of each symbol in order to estimate $n(i)\approx Np(i)\cdot$, giving us:

$$p(\bar{x})\approx\prod_{i=1}^{m}p(i)^{Np(i)}=\prod_{i=1}^{m}\left(2^{\log p(i)}\right)^{Np(i)}=2^{-NH(X)}$$

So if $n(i)=Np(i)$, then the probability of getting this sequence becomes $2^{-NH(X)}$. If N is large enough, we can safetly ignore sequences that don't satisfy $n(i)=Np(i)$ to arbitrary precision, since

that becoms exponentially more unlikely. We call the sequences that satisfy this *typical sequences*. Suppose that leaves us with M such typical sequences left after excluding the *atypical* ones. Summing over $p(\bar{x}) = 2^{-NH(X)}$ yeilds $1 \approx M2^{-NH(X)}$. So there are (approximately $M \approx 2^{NH(X)}$) typical sequences. If we index these, we can therefore send $\log M$ bits to communicate which typical sequence we are trying to send.

3 Entropy as a Measure of Security

Suppose you had a random variable $X = 1, ..., n$. After X is drawn at random, suppose you can ask yes or no questions of the form "is X in the subset $S \subset \{1, ..., n\}$?" How many guesses, on average, will it take you to be able to figure out the value of X?

The answer, as it turns out, is $2^{H(X)}$! The proof of this is left as homework (*hint: how does prefix-free encoding relate to binary trees?*).

Now, imagine that X is a random user's password. We can use entropy as a measure of the uncertainty an attacked might have about this password. Despite the fact that a user *is allowed* to have their password be consist of 20-30 random characters, the *real* distribution of passwords in practice is quite skewed.

Rank	Password	Number of Users with Password (Absolute)
1	123456	290731
2	12345	79078
3	123456789	76790
4	Password	61958
5	iloveyou	51622
6	princess	35231
7	rockyou	22588
8	1234567	21726
9	12345678	20553
10	abc123	17542

Rank	Password	Number of Users with Password (Absolute)
11	Nicole	17168
12	Daniel	16409
13	babygirl	16094
14	monkey	15294
15	Jessica	15162
16	Lovely	14950
17	michael	14898
18	Ashley	14329
19	654321	13984
20	Qwerty	13856

Password Popularity—Top 20

Figure 3: The 20 most common passwords among the 32 million leaked in the 2009 RockYou breach [7].

The Shannon entropy of the full RockYou password distribution has been estimated at only about 21.15 bits [8].

The additive property of entropy also tells us how to build more secure passwords: we can simply append more randomness to the end.

There are a couple limitations to this line of reasoning though.

First, is that there is a misalignement between our previous guesswork model and how guessing passwords is a actually done. Analysis on real password databases has shown that the Shannon entropy does not correlate with difficulty when passwords need to be guessed one-at-a-time [9, p. 44].

Second, is that real password attacks don't take this shape. Let's look at a failure case.

Imagine our userbase is distributed as follows: half the users are secure, and pick a random password from a pool of 2^{128}, and the other half are lazy, and just pick the word "password" as their password.

So X = "password" with probability $\frac{1}{2}$, and each of the 2^{128} random passwords shows up with probability $\frac{1}{2} \cdot 2^{-128} = 2^{-129}$. The entropy of this password distribution is:

$$H(X) = -\underbrace{\frac{1}{2}\log\frac{1}{2}}_{\text{half the users using "password"}} - \underbrace{2^{128} \cdot 2^{-129} \log 2^{-129}}_{\text{half the users picking at random}} = \frac{1}{2} + \frac{129}{2} = 65 \text{ bits}$$

This seems pretty secure! 65 bits is the same uncertainty as a password chosen uniformly from 2^{65} possibilities, which would take an attacker an huge amount of guessing. But for half the users, an attacker can guess the password as "password" on the very first try. If this doesn't work... they can simply pick another user to attack.

The problem is that $H(X)$ averages over the whole distribution, so the enormous uncertainty of the secure half drowns out the fact that the lazy half have no uncertainty at all. An attacker doesn't care about the average; they care about their best guess. This is captured by the *min-entropy*:

$$H_\infty(X) = -\log \max_x p(x) = -\log \frac{1}{2} = 1 \text{ bit}$$

From the attacker's point of view, this userbase is about as secure as a coin flip. Notice also that we can make $H(X)$ as large as we like by growing the pool the secure users pick from, without changing $H_\infty(X)$ at all: the weakest passwords stay exactly as weak. For this reason, min-entropy is the measure NIST uses when assessing the entropy sources that feed cryptographic random number generators [10].

4 Acknowledgements

1. CO487 Course notes
2. Claude for aiding with latex typsetting

References

[1] Nancy Hall. Entropy of a gas. NASA Glenn Research Center, 2021. URL https://www.grc.nasa.gov/www/k-12/airplane/entropy.html. Accessed: 2026-10-25.

[2] GE HealthCare. Entropy monitoring: A valuable tool for guiding delivery of anesthesia. Clinical View, 2025. URL https://clinicalview.gehealthcare.com/quick-guide/entropy-monitoring-valuable-tool-guiding-delivery-anesthesia. Published 15 September 2025. Accessed: 2026-10-25.

[3] Kenneth D. Bailey. *Social Entropy Theory*. State University of New York Press, Albany, NY, 1990. ISBN 978-0-7914-0056-2.

[4] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, Cambridge, 10th anniversary edition, 2010. doi: 10.1017/CBO9780511976667.

[5] Claude E. Shannon. A mathematical theory of communication. *The Bell System Technical Journal*, 27(3):379–423, 1948. doi: 10.1002/j.1538-7305.1948.tb01338.x.

[6] Bertrand Duplantier and Vincent Rivasseau, editors. *Information Theory: Poincaré Seminar 2018*, volume 78 of *Progress in Mathematical Physics*. Birkhäuser, Cham, 2021. ISBN 978-3-030-81480-9. doi: 10.1007/978-3-030-81480-9.

[7] Imperva Application Defense Center. Consumer password worst practices. White paper, Imperva, 2010. URL https://www.imperva.com/docs/gated/WP_Consumer_Password_Worst_Practices.pdf. Accessed: 2026-10-26.

[8] Joseph Bonneau. Statistical metrics for individual password strength. In Bruce Christianson, James Malcolm, Frank Stajano, and Jonathan Anderson, editors, *Security Protocols XX*, volume 7622 of *Lecture Notes in Computer Science*, pages 76–86. Springer, Berlin, Heidelberg, 2012. doi: 10.1007/978-3-642-35694-0_10.

[9] Joseph Bonneau. *Guessing Human-Chosen Secrets*. PhD thesis, University of Cambridge, May 2012. URL https://jbonneau.com/doc/2012-jbonneau-phd_thesis.pdf.

[10] Meltem Sönmez Turan, Elaine Barker, John Kelsey, Kerry A. McKay, Mary L. Baish, and Mike Boyle. Recommendation for the entropy sources used for random bit generation. NIST Special Publication 800-90B, National Institute of Standards and Technology, 2018.